Elegir tecnologías e integrar respuestas — versión ampliada

Para repasar las ideas esenciales antes o después de clase, consulta la versión corta. Esta guía añade ejemplos y criterios para profundizar; no sustituye la explicación del recorrido de UD01.

Elegir backend no consiste en buscar «el mejor lenguaje». Una tecnología puede ser excelente en un contexto y mala decisión en otro. La elección combina tipo de problema, conocimientos del equipo, ecosistema, operación, seguridad, soporte y coste de cambio.

Un framework acelera decisiones repetidas, pero también impone convenciones. Por eso en DWES no interesa aprender una lista de nombres de moda, sino entender qué responsabilidad cumple cada pieza y qué coste introduce.

Idea principal

Antes de elegir una tecnología, hay que poder responder tres preguntas:

1. **Qué problema resuelve.** No es lo mismo servir páginas HTML que exponer una API REST para varios clientes.
2. **Qué delegamos en ella.** Enrutamiento, validación, seguridad, plantillas, acceso a datos, pruebas, despliegue…
3. **Qué coste asumimos.** Aprendizaje, mantenimiento, actualizaciones, dependencias, hosting, rendimiento y migración futura.

La decisión madura no suena a «Spring Boot es mejor que Laravel»; suena a «para este proyecto, con este equipo y estos requisitos, Spring Boot nos conviene por estas razones y nos obliga a cuidar estos costes».

Vocabulario mínimo

- **Lenguaje:** define la sintaxis y la semántica para expresar programas. Ejemplos: Java, PHP, JavaScript, C#.
- **Runtime:** entorno que ejecuta el programa y gestiona recursos básicos. Ejemplos: JVM, intérprete PHP, Node.js, CLR de .NET.
- **Biblioteca:** código reutilizable que la aplicación invoca cuando lo necesita. Ejemplo: una biblioteca para validar emails o generar PDFs.
- **Framework:** define un flujo de trabajo y llama a nuestro código en puntos previstos. Ejemplos: Spring Boot, Laravel, ASP.NET Core.
- **SDK:** conjunto de herramientas, APIs y documentación para una plataforma o servicio. Ejemplo: SDK de AWS o de una pasarela de pago.
- **CMS:** producto orientado a gestionar y publicar contenido. Ejemplos: WordPress, Drupal.

Un mismo elemento puede combinar categorías. Node.js, por ejemplo, suele presentarse como runtime, pero alrededor tiene ecosistema, herramientas y frameworks. Lo importante es explicar qué responsabilidad delega la aplicación.

Lenguaje, runtime, biblioteca y framework no son lo mismo

Ejemplo con Java y Spring Boot:

- El **lenguaje** Java permite escribir clases, records, métodos y tipos.
- La **JVM** ejecuta el bytecode y gestiona memoria, hilos y carga de clases.
- Una **biblioteca** puede aportar una función concreta, como parsear JSON.
- **Spring Boot** organiza la aplicación: arranque, configuración, controladores, inyección de dependencias, validación, seguridad, pruebas y observabilidad.

Ejemplo con PHP y Laravel:

- El **lenguaje** PHP expresa la lógica.
- El **runtime** PHP ejecuta cada petición en el servidor.
- Una **biblioteca** puede aportar una utilidad concreta.
- **Laravel** aporta enrutamiento, controladores, plantillas Blade, Eloquent, validación, middleware, migraciones y comandos.

Confundir estos niveles lleva a frases pobres como «Spring Boot es Java» o «Laravel es PHP». No. Son piezas relacionadas, pero no equivalentes.

Comparación orientativa del módulo

Java + Spring Boot

Fortalezas:

- Tipado estático y buen soporte de refactorización.
- Ecosistema empresarial muy maduro.
- Buenas herramientas de pruebas, observabilidad y documentación.
- Encaja bien con arquitecturas por capas y APIs REST.

Costes:

- Más estructura inicial.
- Más conceptos: contenedor, beans, inyección, configuración, DTOs, validación, seguridad.
- Si se usa sin entender HTTP y diseño, puede parecer «magia».

Papel en DWES: backend principal.

PHP + Laravel

Fortalezas:

- Productividad alta para aplicaciones web.
- Ciclo petición-respuesta muy directo.
- Buen ecosistema para MVC, formularios, autenticación, plantillas y bases de datos.

Costes:

- Muchas convenciones: nombres, rutas, migraciones, modelos, vistas.
- Comportamiento más dinámico que exige disciplina en pruebas y estructura.
- Hay que distinguir lo que hace PHP de lo que hace Laravel.

Papel en DWES: segundo framework completo.

.NET

Fortalezas: plataforma sólida, tipada y con herramientas integradas.

Costes: solapa varios conceptos ya cubiertos con Java y Spring Boot.

Papel en DWES: demostración opcional o comparación puntual.

Node.js

Fortalezas: un solo lenguaje en cliente y servidor, ecosistema enorme.

Costes: modelo asíncrono, gestión de dependencias y variedad de frameworks requieren criterio.

Papel en DWES: fuera del recorrido principal porque ya aparece en el itinerario MERN.

Esta comparación no asigna una calidad absoluta. Obliga a justificar una elección según el contexto.

Framework frente a código propio

Un framework suele proporcionar:

- enrutamiento y ciclo de petición;
- configuración e inyección de dependencias;
- validación de entradas;
- seguridad;
- acceso a datos;
- plantillas o serialización;

- pruebas y herramientas de diagnóstico;
- convenciones compartidas por el equipo.

Eso ahorra trabajo, pero no elimina la responsabilidad del equipo. El framework no decide por nosotros qué recurso existe, qué código HTTP corresponde, qué dato puede ver cada usuario ni cómo se modela una regla de negocio.

El precio es aprender su modelo, actualizarlo y evitar depender de comportamiento que no se comprende. Usar un framework sin entender HTTP, estado, cabeceras y límites produce aplicaciones difíciles de depurar.

Integración con lenguajes de marcas

El servidor puede integrar datos y HTML de varias formas.

HTML estático

El servidor entrega un archivo ya existente. El contenido es igual para todas las peticiones salvo que se cambie el archivo.

Ejemplo conceptual:

```
GET /about.html HTTP/1.1

HTTP/1.1 200 OK
Content-Type: text/html
```

HTML mediante plantilla

El servidor combina una plantilla con un modelo de datos.

Ejemplo conceptual:

```
Plantilla: "Hola, ${nombre}"
Modelo: nombre = "Ada"
Resultado: "Hola, Ada"
```

La plantilla no debe contener consultas complejas ni reglas de negocio. Su responsabilidad es representar un modelo ya preparado.

JSON

El servidor serializa objetos o DTOs para que otro cliente los consuma: navegador con JavaScript, aplicación móvil, otra API o pruebas automatizadas.

```
HTTP/1.1 200 OK
Content-Type: application/json

{"id":7,"title":"Repasar HTTP"}
```

Devolver JSON no convierte automáticamente un endpoint en REST. REST exige pensar en recursos, métodos, estados, cabeceras y representaciones.

Redirección

El servidor no devuelve directamente la representación final, sino una indicación para que el cliente vaya a otra URI.

```
HTTP/1.1 302 Found
Location: /login
```

En APIs JSON suele preferirse devolver estados y cuerpos estructurados. En aplicaciones web tradicionales, las redirecciones son habituales tras formularios.

Cómo se conecta todo en una aplicación real

Una petición típica atraviesa varias piezas:

1. El cliente envía una petición HTTP.
2. El servidor web o contenedor la recibe.
3. El framework decide qué controlador debe atenderla.
4. El controlador valida entrada básica y llama a servicios.
5. La capa de servicio aplica reglas de negocio.
6. La capa de datos consulta o modifica persistencia.
7. La aplicación construye una respuesta: HTML, JSON, redirección o error.

Cada tecnología puede organizar estos pasos de forma distinta, pero el problema de fondo es el mismo. Por eso primero estudiamos cliente-servidor, HTTP y mensajes, y después frameworks.

Criterios de selección

Antes de elegir, responde:

1. ¿Qué requisitos funcionales y de seguridad existen?
2. ¿Qué clientes habrá: navegador, app móvil, otra API, usuarios internos?
3. ¿Qué experiencia real tiene el equipo?
4. ¿Qué soporte, actualizaciones y documentación ofrece el ecosistema?
5. ¿Cómo se prueba, observa y despliega?
6. ¿Qué dependencia crea y cuánto costaría migrar?
7. ¿La herramienta resuelve el problema o solo añade novedad?
8. ¿Qué riesgos aparecen si el proyecto crece?

Errores frecuentes

- Elegir por moda y no por requisitos.
- Pensar que un framework evita aprender HTTP.
- Mezclar lógica de negocio en plantillas.
- Exponer entidades internas directamente como JSON sin pensar en DTOs.
- Creer que «más moderno» significa automáticamente «mejor».
- Ignorar despliegue, logs, configuración y actualizaciones.

Comprobación

Justifica por qué el módulo usa Spring Boot como backend principal y Laravel como segundo framework. La respuesta debe incluir:

1. Un criterio técnico.
2. Un criterio operativo o de mantenimiento.
3. Un criterio didáctico.
4. Una limitación o coste de cada opción.
5. Un ejemplo de cómo ambos pueden producir HTML y JSON.

Esta evidencia trabaja RA1.e, RA1.f y RA1.g.